Bottom up testing

1. Introduction

Bottom-Up Testing is an integration testing approach where testing starts from the **lowest-level modules (classes/functions)** and proceeds upward to higher-level modules.

Higher-level modules are not available initially, so **test drivers are used** to simulate them.

---

2. Objective

- To test **individual modules independently**
- To detect errors at early stage
- To ensure correctness before integration
- To validate class methods and logic

---

3. Concept of Bottom-Up Testing

- Testing starts from **leaf modules**
- Uses **drivers instead of stubs**
- Gradually integrates modules into system

---

4. Driver Concept

A **driver** is a temporary program used to call and test lower-level modules.

Example:

```
int main() {
    cout << add(2,3);
}
```

---

5. Modules Selected from Project

From your system:

- Voter Class
- Candidate Class
- EVM Class
- Complaint Class

(From your file)

---

6. Bottom-Up Testing Strategy

Step 1:

Test lowest-level classes individually

Step 2:

Use **driver program** to call methods

Step 3:

Verify outputs

Step 4:

Integrate modules upward

Test Version Code

```
#include <iostream>
using namespace std;

//
==========================================================
===
//  ◇  LOW LEVEL MODULE (REAL CODE)
//
==========================================================
===

class Voter {
    string name;
    int age;

public:
    Voter(string n, int a) : name(n), age(a) {}

    string getName() { return name; }
    int getAge() { return age; }
```

```
    void castVote() {
        cout << "[Voter] Vote casted by " << name << endl;
    }
};

//
==================================================
===
//  DRIVER (SIMULATES UPPER MODULES LIKE MENU + MAIN)
//
==================================================
===

int main() {

    cout << "=========== BOTTOM-UP TESTING (DRIVER)
===========\n";

    int failed = 0;

    //
=================================================
    //  ◇  TC1: Simulating voterMenu calling Voter
```

```
    //
==================================================
    Voter v("Parth", 20);

    if(v.getName() != "Parth") {
        cout << "TC1 Failed\n";
        failed++;
    }

    if(v.getAge() != 20) {
        cout << "TC2 Failed\n";
        failed++;
    }

    //
==================================================
    //  ◇  TC2: Simulating vote operation (menu action)
    //
==================================================
    cout << "\n[Driver] Calling castVote() (as menu would
do)\n";
    v.castVote();
```

```
    //
==================================================
    //  ◇  FINAL RESULT
    //
==================================================
    if(failed == 0)
        cout << "\n All Bottom-Up Tests Passed\n";
    else
        cout << "\n Tests Failed: " << failed << endl;

    return 0;
}
```

TEST CASE TABLE (WRITE THIS EXACTLY)

Test Case ID	Module	Driver Action	Input	Expected Output	Actual Output	Status
TC1	Voter	Create Object	("Parth", 20)	Object created successfully	Same	Pass
TC2	Voter	getName()	—	"Parth"	"Parth"	Pass

Test Case ID	Module	Driver Action	Input	Expected Output	Actual Output	Status
TC3	Voter	getAge()	—	20	20	Pass
TC4	Voter	castVote()	—	"[Voter] Vote casted by Parth"	Same	Pass

10. Observations

- All modules executed independently
- Outputs are correct
- No runtime errors found
- Modules ready for

Test Case ID	Module	Driver Action	Input	Expected Output	Actual Output	Status
integration						

---

11. Advantages

- Early bug detection
- Strong module-level testing
- No need for stubs

---

12. Limitations

- No early system-level testing
- Requires drivers

Test Case ID	Module	Driver Action	Input	Expected Output	Actual Output	Status
• Integration happens late						